

## Project 2 - Hash Me If You Can!

### Part A: Defining a Custom Hash

Let's start with a hands-on experiment. Right now, our `Item` object doesn't have a hash function defined. Try creating an `unordered_set<Item>` and see what happens—go ahead, I'll wait!

Did you get an error? That's exactly what should happen! The reason is that `unordered_set` relies on *hashing* to organize its elements, but our `Item` class doesn't know how to hash itself yet.

Let's fix that by providing a custom hash function. We'll need to specialize `std::hash<Item>`.

In your `Item.hpp` file, add the following code. Make sure it's *outside* the `Item` class definition:

```
1  template <>
2  struct std::hash<Item> {
3      /**
4       * @brief Computes a hash value for an Item based on
5       * using the standard hash for strings on the Item's name
6       *
7       * @param i The Item to hash
8       * @return Hash value for the Item
9       */
10     size_t operator()(const Item& i) const;
11 };
```

You'll notice something interesting here: the **template is empty**. Instead of using a generic template parameter, we explicitly specify `std::hash<Item>`. This technique is called **template specialization**.

Template specialization lets us provide a custom implementation for specific types within a templated class or function. Think of it this way: the standard library gives us a generic `std::hash` template that works for basic types like `int` or `string`, but it doesn't know how to hash our custom `Item` class. By specializing the template, we're saying "here's exactly how to hash an `Item`" while still keeping the flexibility of the template system for everything else.

Consider the following example:

```
1  // General template
2  template <typename T>
3  class Printer {
4  public:
5      void print(T value) {
6          std::cout << "General:_" << value << std::endl;
7      }
8  };
9
10 // Specialization for `std::string`
11 template <>
12 class Printer<std::string> {
13 public:
14     void print(std::string value) {
15         std::cout << "Specialized_for_string_" << value << std::endl;
16     }
17 };
18
19 int main() {
20     Printer<int> intPrinter;
21     intPrinter.print(42); // Uses general template
22
23     Printer<std::string> stringPrinter;
24     stringPrinter.print("Hello!"); // Uses specialized template
25 }
```

```
26     return 0;
27 }
```

Output:

```
1 General: 42
2 Specialized for string: Hello!
```

**Make sure you understand this before moving forward. The rest of the project relies on this.**

—

Now let's finish up our specialized hash for Item.

In `Item.cpp`, you'll implement the hash function by leveraging the existing `std::hash<std::string>` on the `Item`'s `name_` member. Since strings already know how to hash themselves, we can simply reuse that functionality!

Take a look at the documentation here for guidance: <https://en.cppreference.com/w/cpp/utility/hash>

Don't overthink this—it really is just a one-liner.

```
1  /**
2   * @brief Computes a hash value for an Item based on
3   * using the standard hash for strings on the Item's name
4   *
5   * @param i The Item to hash
6   * @return Hash value for the Item
7   */
8  size_t std::hash<Item>::operator()(const Item& i) const {
9      // Your code here...
10 }
```

## Part B: Custom Comparators

Great! Now we can hash our items. But here's an interesting question: how do we compare them? What makes one item "greater than" another—is it the name? The weight? The type? (For example, how would we compare the Hand of Malenia to the Straight Sword?)

The answer is: it depends on what you're trying to do! That's where **custom comparators** come in. We'll define multiple ways to compare items based on different properties. Specifically, we'll compare items by:

1. Lexicographical order of their names
2. Weight
3. Type

We've already provided you with two of these comparator classes to get you started. Your job is to implement `CompareItemName` in `Compare.cpp`, which compares two `Items` based on the lexicographic order of their names:

```

1  /**
2   * @class CompareItemName
3   * @brief Provides static comparison functions for Item names
4   */
5  class CompareItemName {
6  public:
7      /**
8       * @brief Compares two items by name
9       * @param a First item to compare
10      * @param b Second item to compare
11      * @return true if a's name is less than b's name
12      */
13      static bool lessThan(const Item& a, const Item& b);
14
15      /**
16       * @brief Checks if two items have equal names
17       * @param a First item to compare
18       * @param b Second item to compare
19       * @return true if items have equal names
20       */
21      static bool equal(const Item& a, const Item& b);
22
23      /**
24       * @brief Checks if first item's name is less than or equal to second item's
25       * name
26       * @param a First item to compare
27       * @param b Second item to compare
28       * @return true if a's name is less than or equal to b's name
29       */
30      static bool leq(const Item& a, const Item& b);
31  };

```

**Usage Example** Suppose you have two items:

```

1  Item katana("Katana", 5.5, ItemType::WEAPON);
2  Item sword("Sword", 5.2, ItemType::WEAPON);

```

To compare different properties of `katana` and `sword`:

```

1  CompareItemName::lessThan(katana, sword);    // True, 'katana' < 'sword'
2  CompareItemWeight::lessThan(katana, sword);  // False, 5.5 not < 5.2
3  CompareItemType::equal(katana, sword);       // True, WEAPON == WEAPON

```

## Task 2: Ready? Set? Go!

### A Brief Description of Inventory

Now that we've set up our Item functionality, let's move on to the Inventory class.

In Inventory.hpp and Inventory.cpp, we've provided a base implementation that uses a generic, iterable container under the hood (like std::vector or std::list).

Our goal is to specialize the Inventory class with specific containers to optimize certain operations. For example, some operations might be faster with a hash table, while others benefit from a sorted structure.

One important constraint: **Inventory objects only store items with unique names—no duplicates allowed.** That's why we can hash using just the item's name!

### Part A: Setting up a HashInventory

We've provided HashInventory.hpp and the corresponding signatures in HashInventory.cpp. Your task is to implement these functions according to the specs.

They have been provided to you, but for reference, here are the function signatures from HashInventory.hpp:

```
1  /**
2   * @brief Default constructor for the Inventory template class.
3   *
4   * Initializes an empty inventory with no items, no equipped item,
5   * and zero total weight.
6   *
7   * @tparam Comparator The comparison class for querying items
8   */
9  Inventory();
10
11 /**
12  * @brief Retrieves the value stored in `equipped_`
13  * @return The Item pointer stored in `equipped_`
14  */
15 Item* getEquipped() const;
16
17 /**
18  * @brief Equips a new item.
19  * @param itemToEquip A pointer to the item to equip.
20  * @post Updates `equipped` to the specified item
21  * without deallocating the original.
22  */
23 void equip(Item* itemToEquip);
24
25 /**
26  * @brief Discards the currently equipped item.
27  * @post Deallocates the item pointed to by `equipped`
28  * and sets `equipped` to nullptr, if `equipped` is not nullptr already.
29  */
30 void discardEquipped();
31
32 /**
33  * @brief Retrieves the value stored in `weight_`
34  * @return The float value stored in `weight_`
35  */
36 float getWeight() const;
37
38 /**
39  * @brief Retrieves the value stored in `item_count_`
40  * @return The size_t value stored in `item_count_`
```

```

41  */
42  size_t size() const;
43
44  /**
45   * @brief Retrieves a copy of the container holding inventory items.
46   *
47   * @return Container of items in the inventory
48   */
49  std::unordered_set<Item> getItems() const;
50
51  /**
52   * @brief Attempts to add a new item to the inventory.
53   *
54   * @param target Item to be added to the inventory
55   * @return true if the item was successfully added, false if an item
56   *         with the same name already exists
57   * @post Updates the weight_ member to reflect the new Item pickup
58   */
59  bool pickup(const Item& target);
60
61  /**
62   * @brief Attempts to remove an item from the inventory by name.
63   *
64   * @param name Name of the item to be removed
65   * @return true if the item was successfully removed, false if the
66   *         item was not found in the inventory
67   * @post Updates the weight_ member to reflect removing the Item
68   */
69  bool discard(const std::string& itemName);
70
71  /**
72   * @brief Checks if an item with the given name exists in the inventory.
73   *
74   * @param name Name of the item to search for
75   * @return true if the item exists in the inventory, false otherwise
76   */
77  bool contains(const std::string& itemName) const;
78
79  /**
80   * @brief Destructor for the Inventory class.
81   * @post Deallocates any dynamically allocated resources.
82   */
83  ~Inventory();

```

## Part B: Querying the Inventory

Now that we have items in our inventory, let's add the ability to select all items within a specific range of values. That's what `query()` will do.

**Here's an example.** Suppose we want to repeatedly query for items within a certain weight range.

1) First, we instantiate an `Inventory<CompareItemWeight>`, specifying that our queries will compare items based on weight:

```
1  Inventory<CompareItemWeight> myInventory;
```

2) Next, to query all items weighing between **0.4** and **10.9**, we create two `Items` that represent our lower and upper bounds:

```
1  Item start("LowWeightItem", 0.4, ItemType::None);
2  Item end("HighWeightItem", 10.9, ItemType::None);
```

3) Finally, we call `query()`. Thanks to `CompareItemWeight`, it will compare items using their `weight_` member:

```
1 std::unordered_set<Item> result = myInventory.query(start, end);
```

The result now contains all items with weights between 0.4 and 10.9 (inclusive).

**That's how you use the function. Now it's your turn to implement it:**

```
1 /**
2  * @brief Queries the inventory for items within a specified range.
3  *
4  * Returns a set of items that fall between the start and end items
5  * according to the specified Comparator (inclusive on both ends)
6  *
7  * @param start An Item whose compared property is the lower bound of the query range
8  * @param end An Item whose compared property is the upper bound of the query range
9  * @return std::unordered_set of items within the specified range
10 *
11 * @note Returns an empty set if the end item is less than the start item
12 * @example To select all Items with weights 0.4 to 10.9, we'd setup the class & parameters as :
13 * - this Inventory object is of type Inventory<CompareItemWeight>
14 * - start = Item("some_name", 0.4, ItemType::None)
15 * - end = Item("some_other_name", 10.9, ItemType::None)
16 *
17 */
18 std::unordered_set<Item> query(const Item& start, const Item& end) const;
```

### Task 3: Branching Out to New Methods

Now we'll be making a tree-based implementation of Inventory, but first we have some setup to do.

#### Part A: Traversing the ItemAVL

ItemAVL<Comparator> is an AVL tree whose ordering property relies on the templated Comparator class. For example, an ItemAVL<CompareItemWeight> will order items based on their weight.

However, before we insert an Item, **we need to check that our tree doesn't already contain an item with the same name**. This means we can't rely on our ordering property to search efficiently. Since we want to use no additional memory to track unique names, we'll need to check *every* item in the AVL tree to see if there's a name match.

**NOTE:** This is a non-standard way of implementing an AVL tree, and the efficiency penalty we incur is due to the specific constraints of our problem. Make sure you understand the runtime complexity implications—this isn't how you'd typically use an AVL tree!

To handle this, implement the contains() method in ItemAVL using the public contains() as a wrapper for the private helper contain() method. This pattern should give you a hint about the algorithm to use. Here are the signatures from ItemAVL.hpp:

```
1 private:
2     /**
3      * @brief Internal routine to check if a subtree contains an item with a given name
4      *
5      * @param name The name to check for
6      * @param subroot The root of the subtree to be searched
7      * @return True if a matching Item exists, false otherwise.
8      */
9     bool contains(const std::string& name, const Node* subroot) const;
10 public:
11     /**
12      * @brief Checks if the tree contains an item with a given name
13      * @param name The name to check for
14      * @return True if a matching Item exists, false otherwise.
15      */
16     bool contains(const std::string& name) const;
```

## Part B: Querying a TreeInventory

We had you implement all the functions for HashInventory, so we won't make you do everything again—not much has genuinely changed. Take a look at the code and you'll see what we mean.

**NOTE:** When instantiating a tree-based Inventory, use `Inventory<Comparator, Tree>`, not `Inventory<Comparator, ItemAVL<Comparator>`.

Now let's get to the interesting part. We have an AVL tree whose ordering property is based on the Comparator we pass in as a template—which is exactly what we use to select items in the `query()` function.

This means that unlike in `ItemAVL`'s `contains()` method, we can efficiently search for items based on the property (e.g., `name_`, `weight_`, or `type_`) being used by the Comparator.

So let's take advantage of that!

Here's something to think about: Say we're at a node in the tree. If this node's `Item` is below the minimum of the query range, do we really need to search the left subtree? Remember, everything to the left is *less than* the current `Item`.

Similarly, if we're above the maximum of the query range, do we need to search the right subtree if everything to the right is *greater than* our current `Item`?

That's your hint. The rest is for you to figure out. Your AVL query should be more efficient than  $O(n)$ .

Here's the function signature for convenience. You'll more than likely need a helper function as well, so we've included space for that:

```

1 public:
2 /**
3  * @brief Queries the inventory for items within a specified range.
4  *
5  * Returns a set of items that fall between the start and end items
6  * according to the specified Comparator (inclusive on both ends)
7  *
8  * @param start An Item whose compared property is the lower bound of the query range
9  * @param end An Item whose compared property is the upper bound of the query range
10 * @return std::unordered_set of items within the specified range
11 *
12 * @note Returns an empty set if the end item is less than the start item
13 * @example To select all Items with weights 0.4 to 10.9, we'd setup the class & parameters as
14 * - this Inventory object is of type Inventory<CompareItemWeight>
15 * - start = Item("some_name", 0.4, ItemType::None)
16 * - end = Item("some_other_name", 10.9, ItemType::None)
17 *
18 */
19 std::unordered_set<Item> query(const Item& start, const Item& end) const;
20
21 private:
22 // This is the helper; I won't be providing docs for this one! On you to engineer how you might
23 void queryHelper(const Item& start, const Item& end, const Node* root, std::unordered_set<Item>& result);

```



## Task 4: Time to Time

Now that we've implemented `contains()` and `query()` for our `Inventory` class, it's time to analyze their performance across different data structures. Your task is to measure and compare the runtime of these operations on different inventory implementations:

1. `Inventory<Comparator>` (Uses a **vector**)
2. `Inventory<Comparator, std::list>` (Uses a **linked list**)
3. `Inventory<Comparator, std::unordered_set<Item>` (Uses a **hash table**)
4. `Inventory<Comparator, Tree>` (Uses an **AVL tree**)

For timing your code please use the `crhono` library. Here you can see how it is used: [https://en.cppreference.com/w/cpp/chrono/duration/duration\\_cast](https://en.cppreference.com/w/cpp/chrono/duration/duration_cast) (see the Example code).

**Before you go, some words of advice:** It looks like a lot of timing and testing, but it's not. Remember, all these Inventories *have the same interface, and there's a LOT of repeated steps*. Make sure to create templated functions where you can.

For example, you could define a function like:

```
1 template <Comparator, Container>
2 float timeQuery(Inventory<Comparator, Container> inv, Item start, item end);
```

So you don't have to retype the same code over and over again! It'll make your life easier.

### Part A: Timing `contains()`

Consider:

```
1 n = { 1000, 2000, 4000, 8000 }
```

For each `Inventory` type listed above (using `CompareItemName` as the `Comparator`) and each value of `n`:

1. Set up the test data:
  - a. Create a new `ItemGenerator` with seed 42.
  - b. Generate and insert `n` random items into the inventory using `ItemGenerator::randomItem()`.
  - c. Use `ItemGenerator::randomUsedName()` to select 100 random item names from the inventory. Store these names in a vector called `contained`.
  - d. Using the same generator, create 100 additional random items with `ItemGenerator::randomItem()`. Do *not* add these items to the inventory. Instead, record their names in a vector called `missing`.
2. Measure the execution time of `contains()` for each name in both `contained` and `missing` (200 total calls). For each call, start the timer immediately before `contains()` and stop it immediately after. Record the execution time in milliseconds and add it to a running total.
3. Calculate the average `contains()` execution time by dividing the total elapsed time by 200.
4. Record your results in a table.

### Part B: Timing `query()`

You will test the performance of `query()` across different inventory types using the following input sizes:

```
1 n = { 1000, 2000, 4000, 8000 }
```

For each inventory type and each value of `n`, you'll run two sets of tests:

#### Test 1: `CompareItemName`

1. Create an Inventory with Comparator set to CompareItemName.
2. Create a new ItemGenerator with seed 42. Insert n randomly generated items into the inventory.
3. Perform the following steps 10 times:
  - a. Get two randomly used names from the generator and assign them to Items named start and end. If necessary, swap them to ensure start < end.
  - b. Measure the execution time of query(start, end). Only the query() call itself should be timed—start the timer immediately before the call and stop it immediately after. Add the elapsed time to a running total.
4. Calculate the average query() time by dividing the total by 10. Record this value.

### Test 2: CompareItemWeight

1. Create an Inventory with Comparator set to CompareItemWeight.
2. Create a new ItemGenerator with seed 42. Insert n randomly generated items into the inventory.
3. Perform the following steps 10 times:
  - a. Call randomFloat from the generator with MIN\_WEIGHT and MAX\_WEIGHT as parameters. Create an Item called start with this weight value, and create another Item called end whose weight\_ is 0.1 units higher than start.
  - b. Measure the execution time of query(start, end). Add the elapsed time to a running total.
4. Calculate the average query() time by dividing the total by 10. Record this value.

*Optional:* You may also test CompareItemType inventories using ItemGenerator::randomType(), but this is not required.

### Part C: Reporting Your Findings

1. Create two tables to present your timing results: one for Part A (contains()) and one for Part B (query()).
2. Write a brief analysis of your findings (approximately one paragraph). Address the following:
  - Were the results what you expected? Why or why not?
  - What explains the performance differences you observed?
  - When would you choose a hash-based inventory over a tree-based inventory, and vice versa?

Focus on clear, concise explanations rather than length.

## Submission Guidelines

You'll be submitting:

1. A report containing your average execution times and analysis (see instructions above)
2. `Item.cpp`
3. `Item.hpp`
4. `Compare.cpp`
5. `Compare.hpp`
6. `HashInventory.cpp`
7. `HashInventory.hpp`
8. `ItemAVL.cpp`
9. `ItemAVL.hpp`
10. `TreeInventory.cpp`
11. `TreeInventory.hpp`

Note that you are given an initial code for all classes (this code will be updated by you). You are also given an empty `main.cpp` file as well as a `Makefile`. This is where you can do your testing and produce your timing results. But you will not be submitting your `main.cpp` file or `Makefile`. These files are for you to use, compile and test your classes. Note as always that you have to upload using the exact same file names (including extensions and capital letters).